Universidade Maurício de Nassau
Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Documento da Sprint 11 — Preparação para entrega

Kaian Guthierry da Silva — 01617843
Nivaldo José de Arruda Filho — 01486768
Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. O que esta Sprint entrega
2. Manual do Usuário (Anexo A)
3. Manual Técnico (Anexo B)
4. Documentação final consolidada
5. Revisão completa do código
6. Correções e melhorias
7. Organização do GitHub
8. Testes depois da revisão
9. Pendências e próximos passos
10. Repositório

1. O que esta Sprint entrega

A Sprint 11 deixa o BuscaPet pronto para ser entregue e apresentado: documentado para quem usa e para quem desenvolve, com o código revisado (inclusive a segurança das bibliotecas) e o repositório organizado. A versão do sistema continua `1.0.0-rc.1`; a versão final (`1.0.0`) sai no code freeze da Sprint 12.

Entrega exigida	Onde está
Manual do Usuário	Anexo A deste documento · `BuscaPet-Docs/manuais/` (PDF e Markdown)
Manual Técnico	Anexo B deste documento · `BuscaPet-App/docs/manual-tecnico.md`
Documentação final consolidada	`BuscaPet-Docs/documentacao-final.md` (seção 4)
Revisão completa do código	seção 5
Organização do GitHub	seção 7
Correções e melhorias	seção 6
Pendências e próximos passos	seção 9

2. Manual do Usuário (Anexo A)

Escrito para quem nunca viu o sistema, com 22 imagens das telas. Organizado por perfil, cobre: apresentação e objetivo, público-alvo, acesso e autenticação (cadastro, login, recuperação de senha, menu da conta), adoção passo a passo, registro de animal perdido ou avistado e os alertas, doação de itens e em dinheiro (PIX) com a meta do mês, o que o abrigo, o apoiador e a administração fazem (cadastros, consultas, relatórios e exportações), e uma seção de mensagens, validações e perguntas frequentes com o significado de cada mensagem e o que fazer.

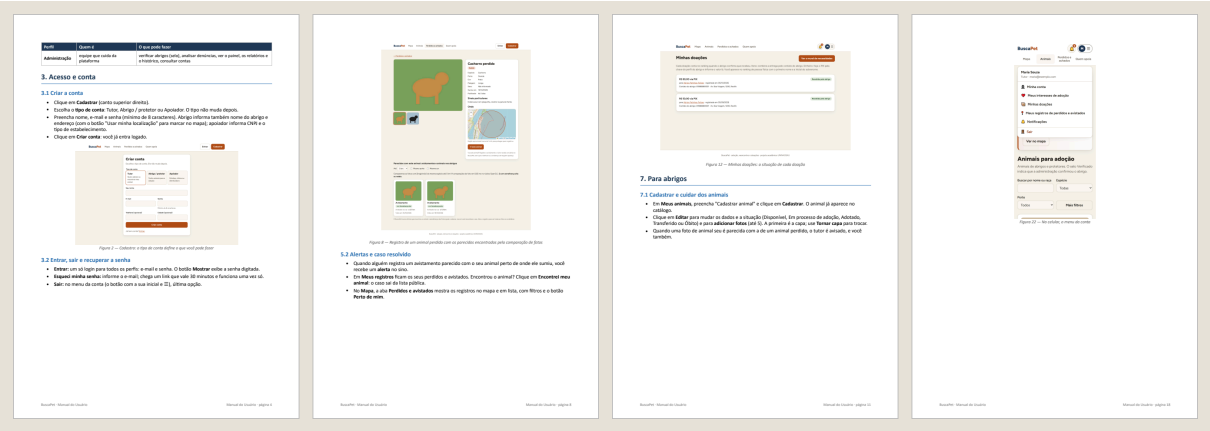


Figura 1 — Algumas páginas do Manual do Usuário (íntegra no Anexo A)

3. Manual Técnico (Anexo B)

Fica no repositório do código (`docs/manual-tecnico.md`), ao lado do que descreve, e foi convertido em PDF para este documento. Cobre os 11 itens pedidos, em 15 seções:

- **Arquitetura** (diagrama dos três serviços e do banco), **tecnologias e bibliotecas** com as versões e a **organização dos módulos**.
- **Banco de dados** (19 tabelas, migrações 0001 a 0005, convenções), **requisitos de ambiente**, **instalação**, **execução**.
- **Variáveis de ambiente** sem nenhuma credencial (só exemplos), **APIs e integração** (a API própria, o núcleo e os serviços externos).
- **Componente de processamento:** como o núcleo OpenCL extrai os descritores, compara em paralelo e o desempenho medido (até 3,8 vezes mais rápido nos 4 núcleos).

- **Segurança**, **testes e CI**, **publicação em produção** (passo a passo para a Sprint 12), **manutenção** (o que rodar ao mudar rota ou modelo) e **limitações conhecidas**, justificadas.

4. Documentação final consolidada

A documentação produzida ao longo das Sprints foi revisada contra a versão atual. O resultado é o `documentacao-final.md` (https://github.com/Equipe-BuscaPet/BuscaPet-Docs), que indica o documento vigente de cada assunto (requisitos, casos de uso, arquitetura, diagramas, modelo de dados, API, regras, testes), registra as decisões que mudaram durante o desenvolvimento e cruza os 41 requisitos funcionais com o que foi implementado.

Resultado da revisão	Quantidade
Requisitos funcionais atendidos como no escopo	37 de 41
Atendidos com mudança registrada e justificada (RF-06, RF-11, RF-15, RF-37)	4 de 41
Decisões de projeto que mudaram, com o porquê	9
Guias de processo e índice corrigidos (Redis, `.env.example` e `CONTEXTO.md` que saíram do repositório)	3
Documentos gerados do código e conferidos por teste (guia da API, MER, schema.sql)	3

5. Revisão completa do código

A revisão combinou ferramentas automáticas e leitura do código, módulo a módulo:

Verificação	Ferramenta	Resultado
Estilo e erros comuns (API)	ruff	sem avisos
Código sem uso	vulture + leitura	configuração do Redis nunca usada (removida); 3 itens do modelo previstos sem tela (mantidos e documentados)
TODO/FIXME, `print` e `console.log` esquecidos	busca no código	nenhum
Tratamento de erros (`except` amplos)	leitura	4 casos, todos corretos (desfazem a operação e repassam o erro, ou escondem detalhes do banco)
Segredos no código e no histórico do Git	busca no histórico inteiro	nenhum segredo real; só valores de exemplo
Vulnerabilidades nas bibliotecas da API	pip-audit	65 alertas em 5 pacotes → 0
Vulnerabilidades nas bibliotecas da interface	npm audit	2 no React Router → 0; resta 1 do servidor de desenvolvimento (justificado)
Imagens Docker e publicação	leitura + teste da imagem	3 problemas corrigidos (seção 6)
Redirecionamento depois do login	teste de ponta a ponta	1 falha de segurança encontrada e corrigida

5.1 O que a revisão encontrou

Problema	Risco	Correção
O login aceitava `?voltar=/\site-externo`: o navegador trata `/\` como `//` e levava a pessoa para outro site	redirecionamento aberto (usado em golpes de phishing)	o retorno só aceita caminhos internos; teste com 3 variantes de ataque
`python-jose` (login JWT) com falhas conhecidas, algumas sem correção	falsificação de token	troca pelo PyJWT

Problema	Risco	Correção
Starlette, Pillow e React Router com falhas conhecidas	negação de serviço, imagem maliciosa, redirecionamento	atualização (FastAPI 0.158, Pillow 13, React Router 7)
Sem `.dockerignore`: o `.env` com as senhas iria para dentro da imagem da API	vazamento de credenciais	`.dockerignore` na API e na interface; conferido que a imagem não tem `.env` nem `.venv`
Chave que assina os logins com valor de exemplo como padrão	qualquer pessoa forjaria um login em produção	a API se recusa a subir num banco remoto com o valor de exemplo
Imagem da API em modo de desenvolvimento (`--reload`), como root e com porta fixa	desempenho e segurança em produção	modo de produção, usuário sem privilégios, porta `$PORT`; o `--reload` ficou só no Docker Compose
Redis e RQ configurados e nunca usados	serviço a mais para manter e pagar	removidos do código, do Docker Compose e da documentação

6. Correções e melhorias

Commit	O que mudou
`b6506e9`	Segurança: PyJWT, bibliotecas atualizadas e a trava do `JWT_SECRET`
`05fc74e`	Login: retorno só para caminhos internos; React Router 7
`bc382b8`	Produção: imagem da API sem segredos nem `--reload`, usuário próprio, porta `$PORT`; Redis removido; regras de rota para hospedagem estática
`6c51a45`	Guia da API compatível com o FastAPI novo
`e6f4183`	Manual técnico
`89053d6`	Diagrama de arquitetura em Mermaid no manual técnico

Também foram preparadas, para a publicação da Sprint 12, as regras que fazem as páginas internas abrirem em hospedagem estática (`vercel.json` e `_redirects`) e a seção de publicação do manual técnico.

7. Organização do GitHub

Item verificado	Situação
Organização das pastas e arquivos	`api/`, `frontend/`, `nucleo-opencl/`, `docs/`; nenhum arquivo gerado ou temporário versionado
Código-fonte atualizado	Pull Requests #20 a #23 empilhados, para integração na ordem
README com instruções de execução	sim, com os 9 itens exigidos e o link para o manual técnico
Documentação técnica disponível	manual técnico, guia da API, MER, schema.sql, README do núcleo
Arquivos de configuração e dependências	`requirements.txt`, `package.json`, Dockerfiles, `docker-compose.yml`, `.dockerignore`, CI
Histórico de commits organizado	commits pequenos e em português, um assunto por commit, por Pull Request
Ausência de arquivos desnecessários e informações sensíveis	`.env` fora do Git e das imagens; histórico verificado; 19 branches já integradas apagadas no BuscaPet-App e 8 no BuscaPet-Docs
Identificação dos integrantes e informações gerais	README dos dois repositórios; descrição e tópicos no GitHub

8. Testes depois da revisão

Com as bibliotecas atualizadas, toda a suíte foi executada de novo: **249 testes da API** passando, com **94%** de cobertura, e **91 testes de ponta a ponta** passando no navegador, no computador e no celular.

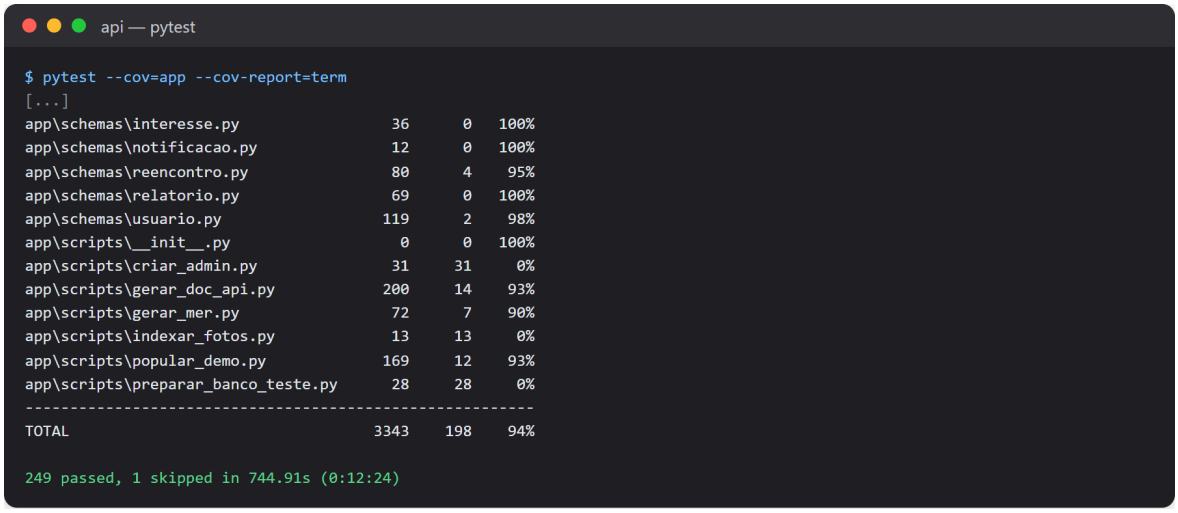


Figura 2 — Execução final da suíte da API (trecho final)

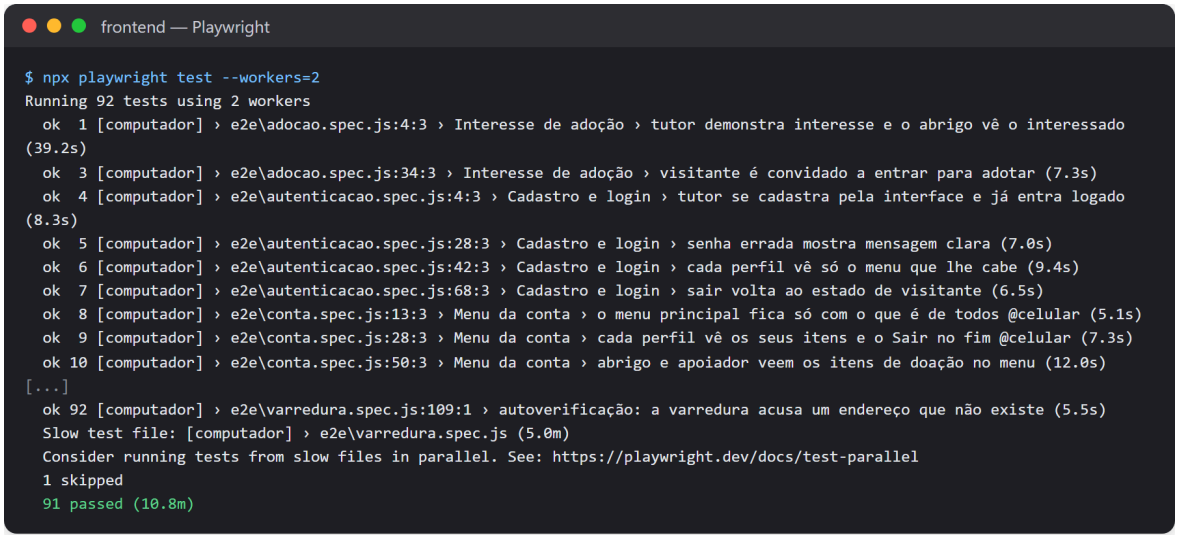


Figura 3 — Execução final dos testes de ponta a ponta (trecho final)

9. Pendências e próximos passos

A Sprint 12 e a publicação em produção ficam com o líder da equipe. O manual técnico (seção 13) e um guia de passagem interno trazem o passo a passo.

Pendência	Como e quando será resolvida
Integrar os Pull Requests #20 a #23 (código) e #9 e #10 (documentação)	nesta ordem, com a CI verde
Migração 0005 no banco compartilhado (Supabase)	`alembic upgrade head`, logo depois da integração
Publicar banco, API, núcleo e interface	até 28/11, com disco persistente para as fotos e memória para o núcleo
Fotos reais nos dados da apresentação	antes do ensaio, para a busca por semelhança ser convincente
Sprint 12	code freeze com a tag `v1.0.0`, os dois vídeos e o ensaio completo da apresentação

Pendência	Como e quando será resolvida
Ao final do projeto	trocar todas as senhas e chaves usadas durante o desenvolvimento

10. Repositório

- Código (BuscaPet-App): https://github.com/Equipe-BuscaPet/BuscaPet-App — Pull Request #23 desta Sprint
- Documentação (BuscaPet-Docs): https://github.com/Equipe-BuscaPet/BuscaPet-Docs — manuais em `manuais/` e `documentacao-final.md`

Universidade Maurício de Nassau
Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Manual do Usuário — versão 1.0

Kaian Guthierry da Silva — 01617843
Nivaldo José de Arruda Filho — 01486768
Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. O que é o BuscaPet
2. Para quem é
3. Acesso e conta
4. Adotar um animal
5. Perdi ou encontrei um animal
6. Doar
7. Para abrigos
8. Para apoiadores
9. Para a administração
10. Mensagens, validações e situações comuns

1. O que é o BuscaPet

O BuscaPet é um site, que também pode ser instalado no celular como aplicativo, que junta num só lugar três formas de ajudar os animais de Recife e região:

- **Adoção:** abrigos e protetores mostram os animais disponíveis; quem quer adotar encontra, conhece e demonstra interesse.
- **Perdidos e achados:** quem perdeu um animal ou viu um animal na rua registra com foto e local. O BuscaPet compara as fotos e mostra os animais mais parecidos, e avisa quando aparece um novo.
- **Doações:** abrigos verificados publicam o que precisam (ração, remédios, dinheiro para as contas do mês) e quem quer ajudar doa. As doações confirmadas aparecem no ranking de quem apoia.

O BuscaPet **nunca afirma** que encontrou o seu animal: ele mostra os mais parecidos, do mais para o menos parecido, e quem reconhece é você. Também **não recebe pagamentos**: doação em dinheiro é feita por PIX, direto do seu banco para o abrigo.

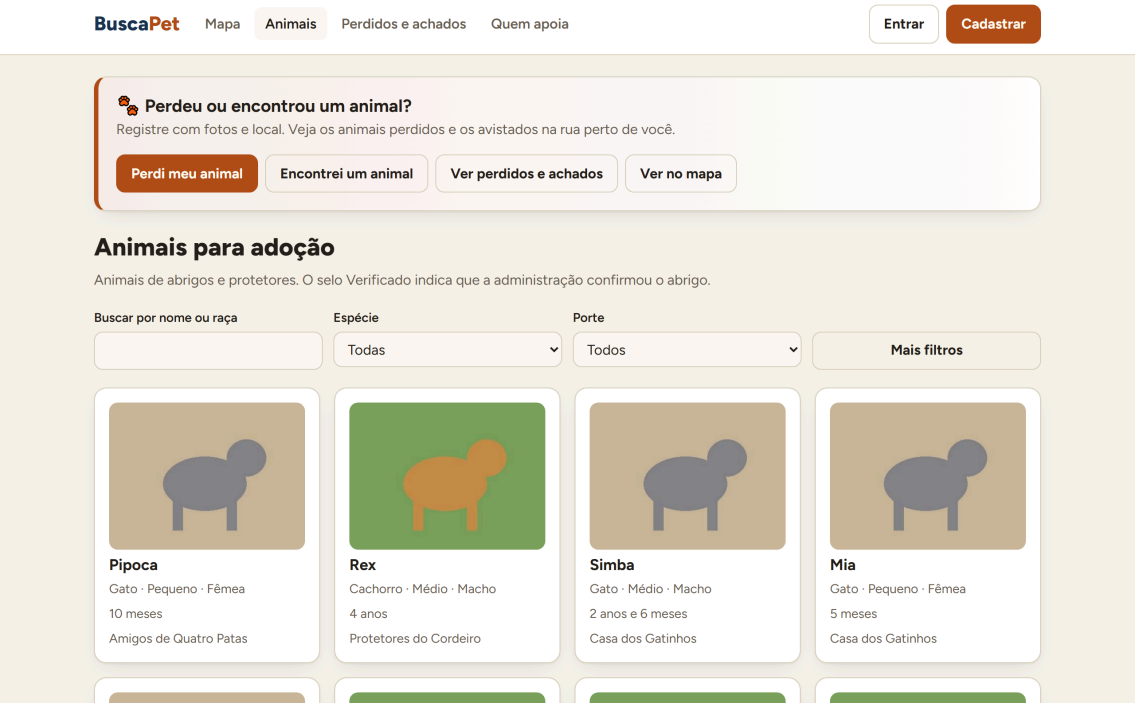


Figura 1 — Página inicial: animais para adoção e o atalho para perdidos e achados

2. Para quem é

Perfil	Quem é	O que pode fazer
Visitante	qualquer pessoa, sem conta	ver animais, mapa, perdidos e achados, ranking e mural de necessidades; preencher um registro de perdido ou avistado (a conta só é pedida na hora de publicar)
Tutor	pessoa que quer adotar, perdeu ou viu um animal, ou quer doar	demonstrar interesse em adotar, registrar perdidos e avistados, receber alertas, doar itens e dinheiro, aparecer no ranking de pessoa física
Abrigo	abrigo ou protetor independente	cadastrar animais com fotos, responder interessados, publicar necessidades e a meta do mês (com o selo Verificado), confirmar doações, ver relatórios
Apoiador	empresa: pet shop, clínica veterinária ou distribuidora	doar pelo mural e aparecer no ranking da sua categoria, com perfil público

Perfil	Quem é	O que pode fazer
Administração	equipe que cuida da plataforma	verificar abrigos (selo), analisar denúncias, ver o painel, os relatórios e o histórico, consultar contas

3. Acesso e conta

3.1 Criar a conta

- Clique em **Cadastrar** (canto superior direito).
- Escolha o **tipo de conta**: Tutor, Abrigo / protetor ou Apoiador. O tipo não muda depois.
- Preencha nome, e-mail e senha (mínimo de 8 caracteres). Abrigo informa também nome do abrigo e endereço (com o botão "Usar minha localização" para marcar no mapa); apoiador informa CNPJ e o tipo de estabelecimento.
- Clique em **Criar conta**: você já entra logado.

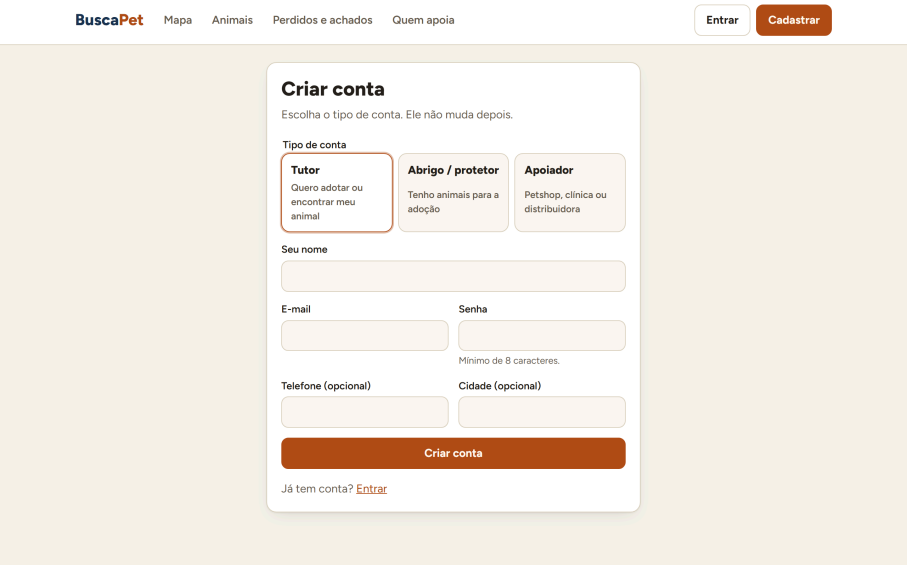


Figura 2 — Cadastro: o tipo de conta define o que você pode fazer

3.2 Entrar, sair e recuperar a senha

- **Entrar:** um só login para todos os perfis: e-mail e senha. O botão **Mostrar** exibe a senha digitada.
- **Esqueci minha senha:** informe o e-mail; chega um link que vale 30 minutos e funciona uma vez só.
- **Sair:** no menu da conta (o botão com a sua inicial e ☰), última opção.

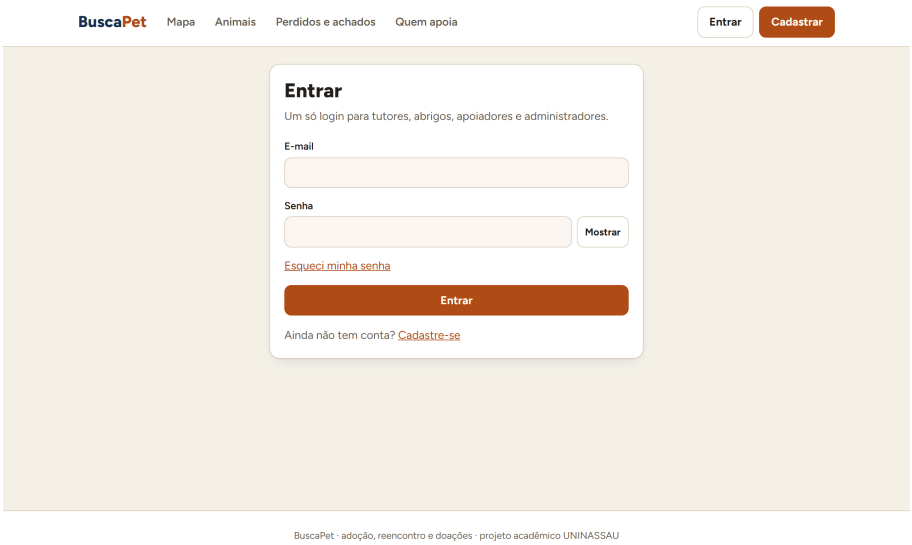


Figura 3 — Tela de entrada

3.3 Menu da conta e Minha conta

O botão com a sua inicial abre o **menu da conta**, com os atalhos do seu perfil. Em **Minha conta** você altera os seus dados (e, no abrigo, o perfil público e a chave PIX), troca a senha ou exclui a conta. O **sino** mostra quantos avisos você ainda não leu.

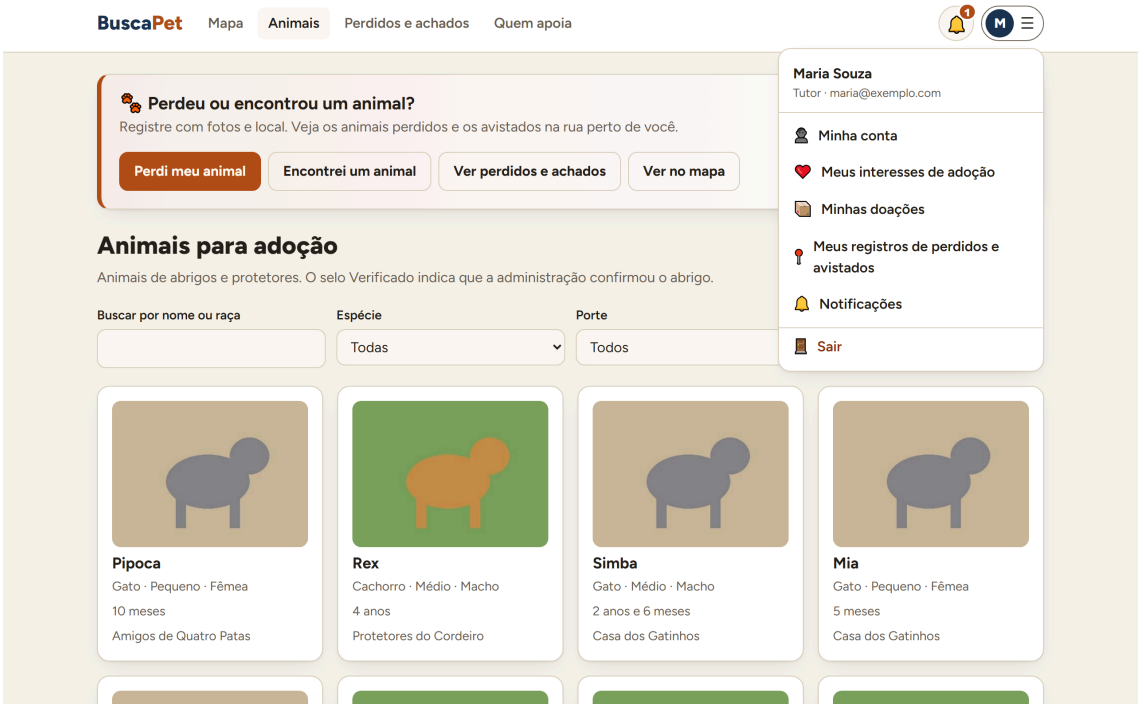


Figura 4 — Menu da conta do tutor

4. Adotar um animal

- Na página **Animais**, use a busca e os filtros (espécie, porte, sexo, idade e outros em "Mais filtros"). O selo **Verificado** indica que a administração conferiu o abrigo.
- Clique no animal para abrir a ficha: fotos, características, temperamento e o abrigo.
- Clique em **Tenho interesse** e, se quiser, escreva uma apresentação (onde mora, rotina, outros animais).
- O abrigo recebe o aviso e responde. Acompanhe em **Meus interesses de adoção**: Aguardando resposta → Em conversa → Aprovado (ou Não seguiu). Com o interesse registrado, você vê o telefone

e o WhatsApp do abrigo.

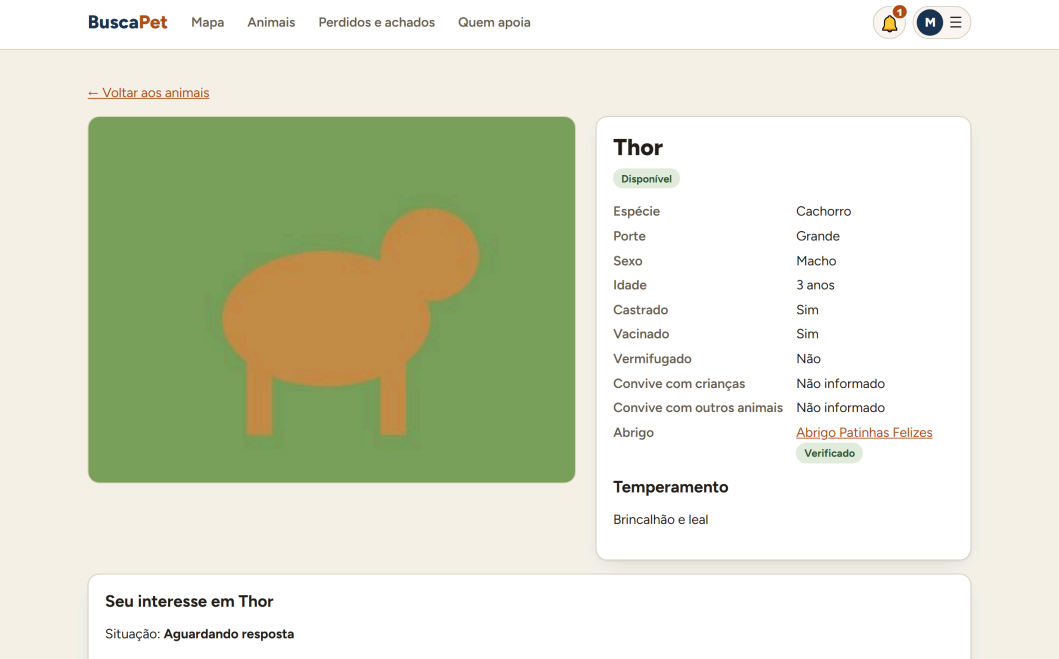


Figura 5 — Ficha do animal com o botão Tenho interesse

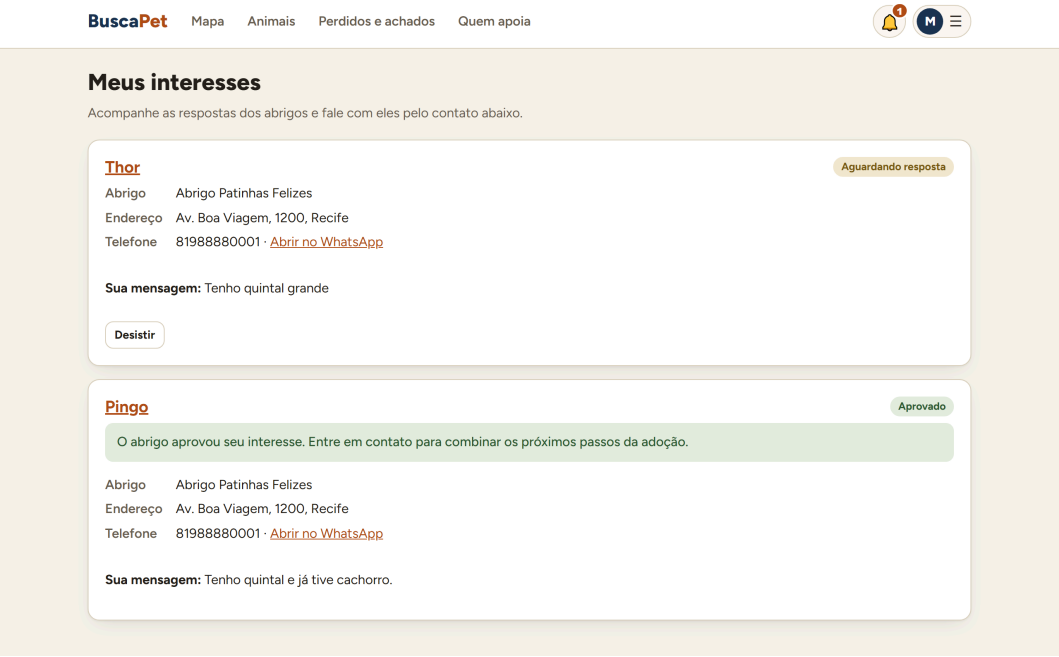


Figura 6 — Meus interesses: a situação de cada pedido de adoção

5. Perdi ou encontrei um animal

5.1 Registrar

Em **Perdidos e achados**, escolha **Perdi meu animal** ou **Encontrei um animal**. O registro tem 5 etapas: **Fotos → Características → Local e data → Revisar → Resultados**. Você pode preencher tudo sem conta; o login é pedido na etapa Revisar, e as fotos continuam lá.

- **Fotos:** de 1 a 5, JPEG, PNG ou WebP, até 8 MB cada. Prefira o corpo inteiro, com boa luz e de ângulos diferentes: é isso que a comparação usa. A localização gravada dentro da foto pelo celular é apagada no envio.
- **Características:** espécie, porte, cor, pelagem e sinais particulares (coleira, cicatriz, mancha).

- **Local e data:** marque no mapa onde o animal sumiu ou foi visto. O público vê só um ponto **aproximado**, para proteger o seu endereço; o local exato fica só para você.
- **Resultados:** os animais mais parecidos (avistados e de abrigos), com o grau de semelhança (alta, média ou baixa) e a distância.

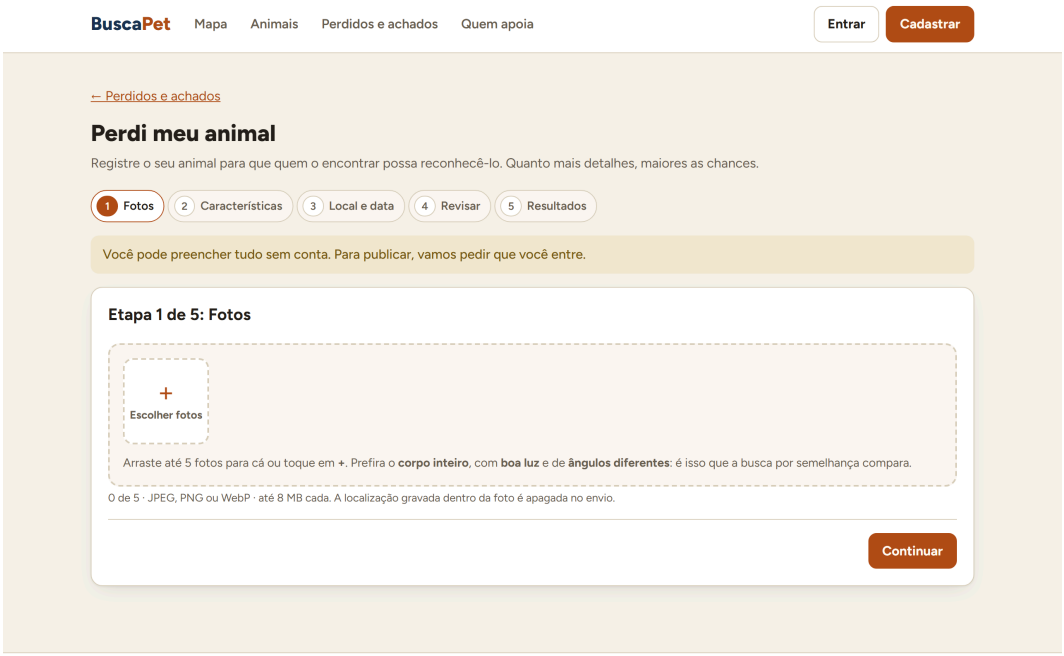


Figura 7 — Perdi meu animal: etapa 1 de 5

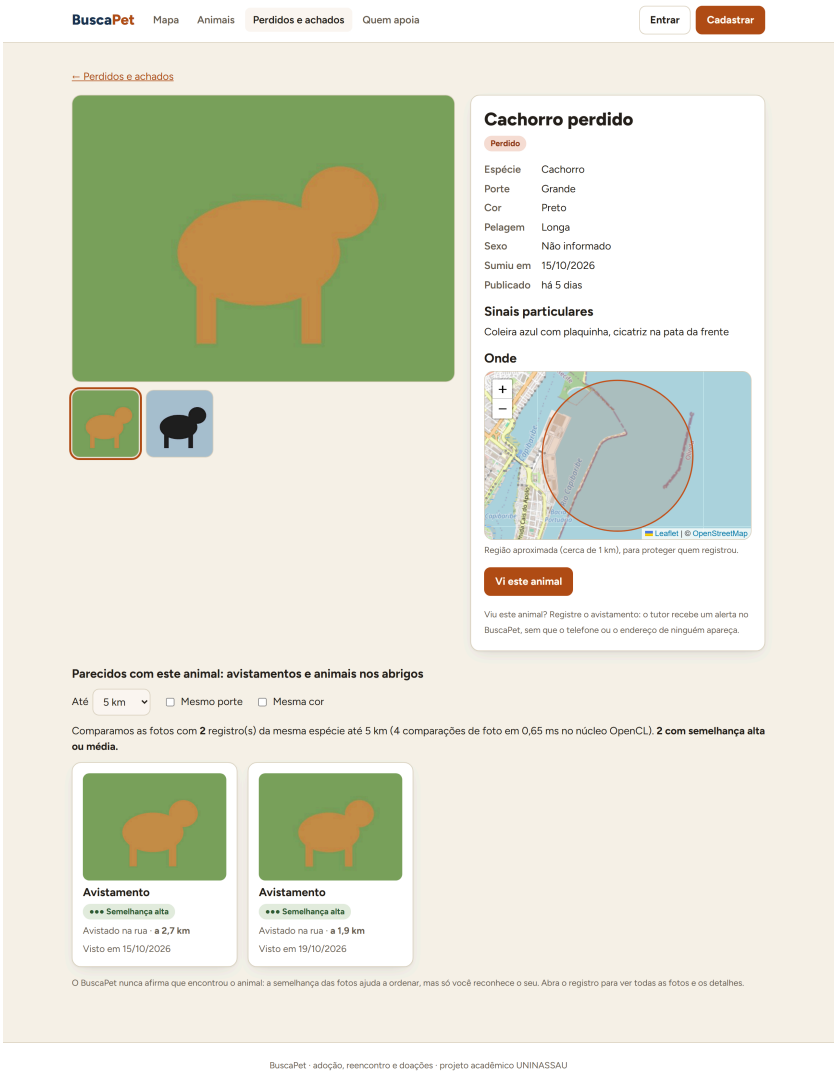


Figura 8 — Registro de um animal perdido com os parecidos encontrados pela comparação de fotos

5.2 Alertas e caso resolvido

- Quando alguém registra um avistamento parecido com o seu animal perto de onde ele sumiu, você recebe um **alerta** no sino.
- Em **Meus registros** ficam os seus perdidos e avistados. Encontrou o animal? Clique em **Encontrei meu animal**: o caso sai da lista pública.
- No **Mapa**, a aba **Perdidos e avistados** mostra os registros no mapa e em lista, com filtros e o botão **Perto de mim**.

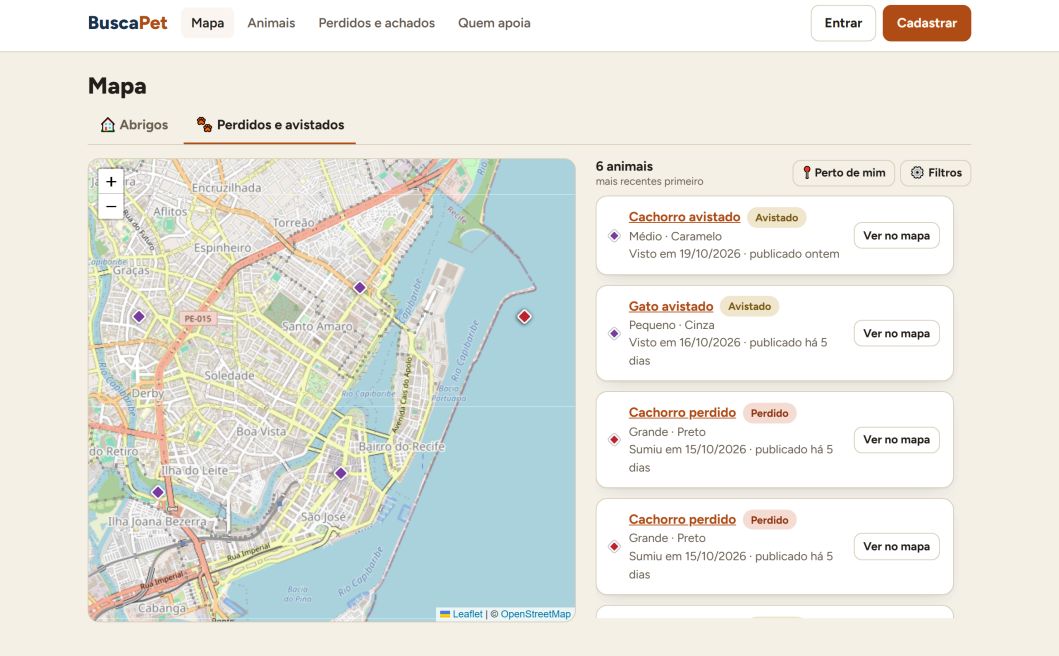


Figura 9 — Mapa: perdidos e avistados

6. Doar

6.1 Doar itens

- No **Quem apoia** (mural) ou no perfil do abrigo, escolha uma necessidade, informe a quantidade e clique em **Quero doar**.
- Combine a entrega pelo contato do abrigo, que aparece em **Minhas doações**.
- Quando o abrigo confirmar que recebeu, a doação conta para a necessidade e para o ranking.

6.2 Doar dinheiro (PIX)

- No perfil de um abrigo verificado aparecem a **meta do mês** (quanto ele precisa e para quê) e a **chave PIX**, com o botão **Copiar**.
- Faça o PIX pelo app do seu banco. O BuscaPet não recebe nem repassa dinheiro.
- Volte ao perfil e, em "Já fez o PIX? Informe o valor", digite o valor (ex.: 50,00) e clique em **Informar doação**.
- O abrigo confere a conta e confirma; aí a meta avança e a doação entra no ranking.

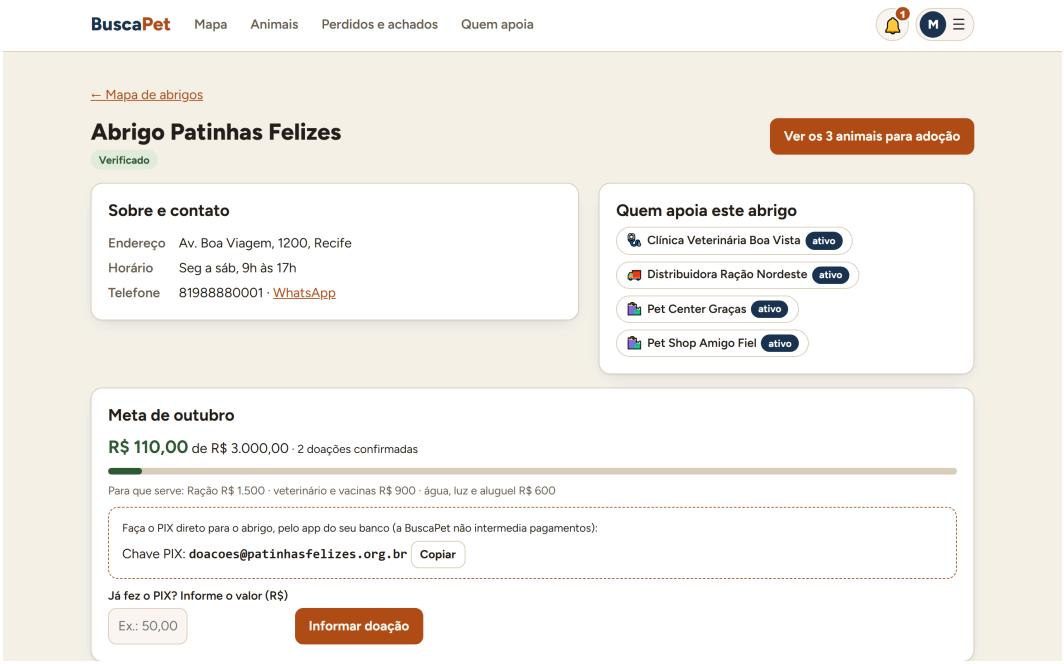


Figura 10 — Perfil do abrigo: meta do mês, chave PIX e o campo para informar a doação

6.3 Ranking de quem apoia

Pontos: 10 por doação confirmada (itens ou dinheiro) + 10 por mês seguido doando; manter a constância vale tanto quanto doar mais. Há uma aba por categoria (petshop, clínica, distribuidora e **pessoa física**), no mês e no ano. Pessoas físicas aparecem só com o primeiro nome e a inicial do sobrenome.

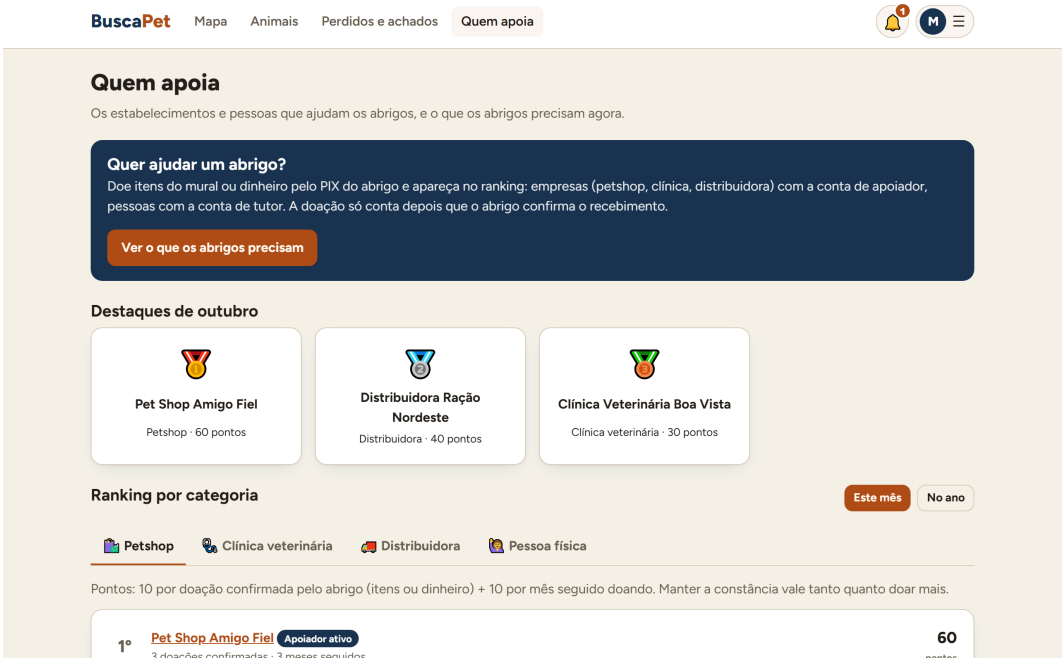


Figura 11 — Quem apoia: destaques do mês e ranking por categoria

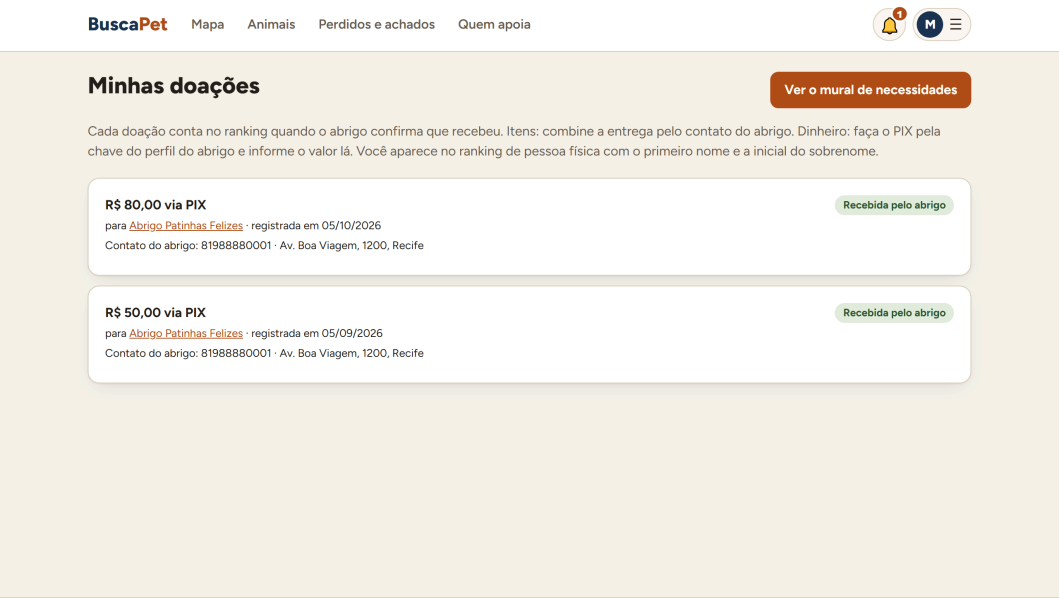


Figura 12 — Minhas doações: a situação de cada doação

7. Para abrigos

7.1 Cadastrar e cuidar dos animais

- Em **Meus animais**, preencha "Cadastrar animal" e clique em **Cadastrar**. O animal já aparece no catálogo.
- Clique em **Editar** para mudar os dados e a situação (Disponível, Em processo de adoção, Adotado, Transferido ou Óbito) e para **adicionar fotos** (até 5). A primeira é a capa; use **Tornar capa** para trocar.
- Quando uma foto de animal seu é parecida com a de um animal perdido, o tutor é avisado, e você também.

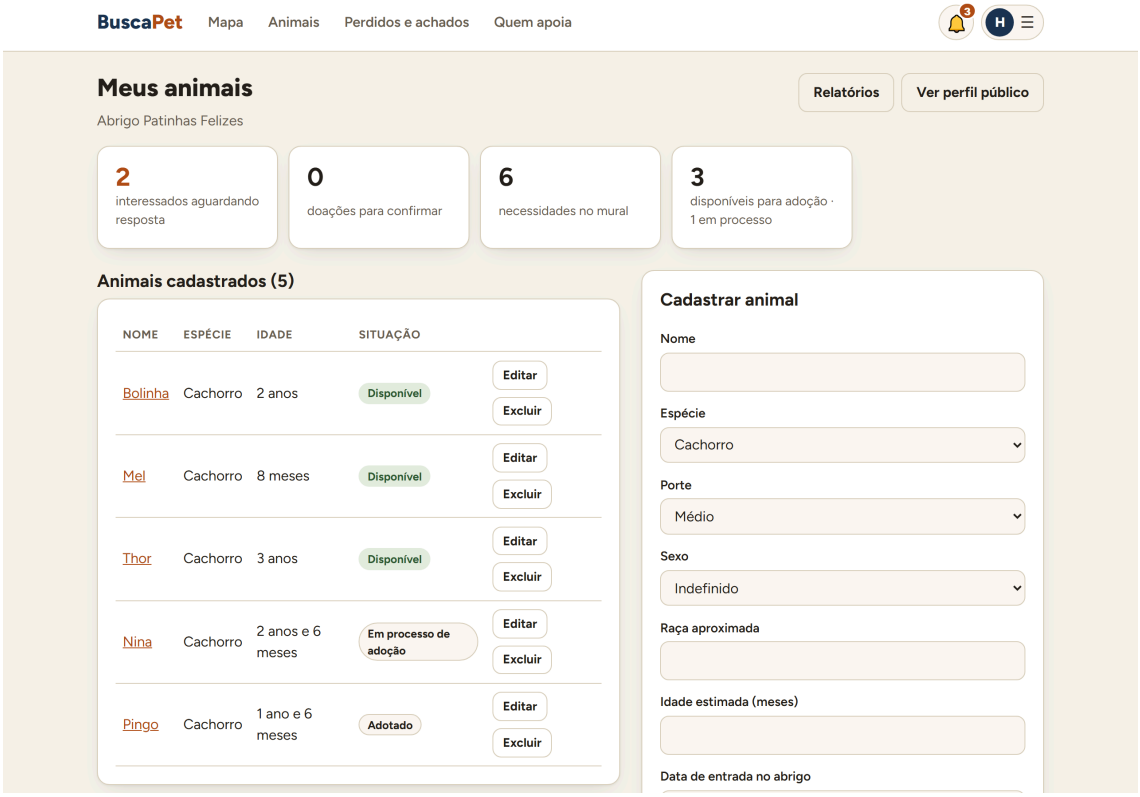


Figura 13 — Meus animais: resumo do dia, lista e cadastro

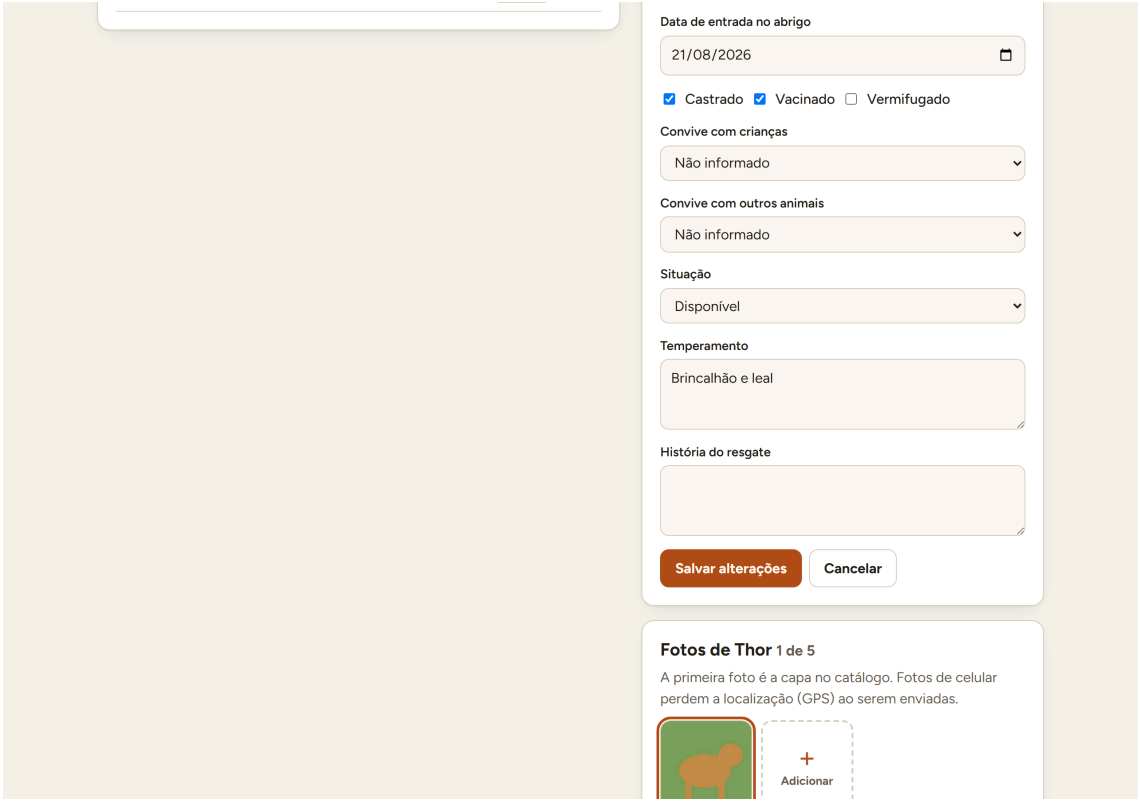


Figura 14 — Edição do animal com as fotos

7.2 Responder interessados

Em **Interessados**, cada pedido mostra o animal, a apresentação e o contato da pessoa. Use **Iniciar conversa**, **Aprovar** ou **Recusar**; a pessoa é avisada a cada mudança. Ao aprovar, o animal passa para "em processo de adoção".

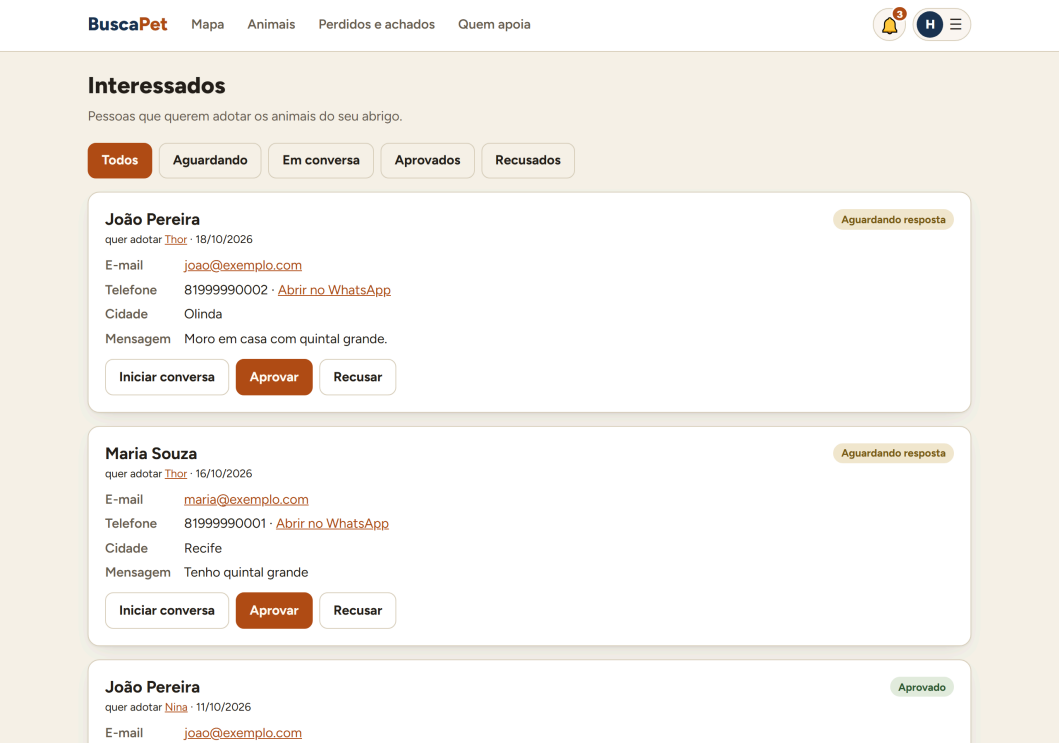


Figura 15 — Interessados: responder cada pedido

7.3 Necessidades, meta do mês e doações

- O **selo Verificado** é concedido pela administração depois de conferir o abrigo. Com ele, o abrigo publica no mural, mostra a chave PIX e recebe doações.
- Em **Necessidades e doações**: defina a **meta do mês** (valor e para que serve), publique necessidades com a urgência, marque como atendida ou exclua.
- Em **Doações para confirmar**, clique em **Confirmar recebimento** (itens) ou **Confirmar: o PIX chegou** (dinheiro) só quando receber de verdade. **Não recebi** recusa. As anteriores ficam no histórico.
- A chave PIX é cadastrada em **Minha conta**.

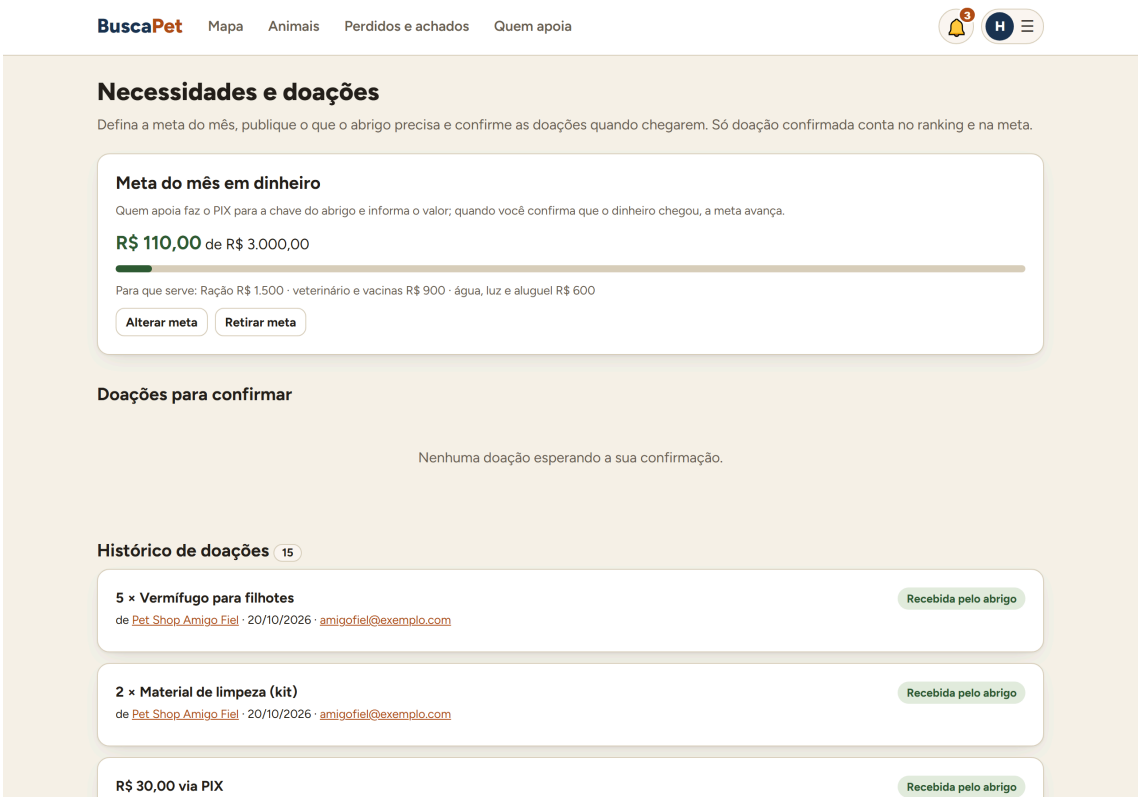


Figura 16 — Necessidades e doações: meta do mês, pendentes e histórico

7.4 Relatórios

Em **Relatórios**, escolha o período (30 dias, 90 dias, este ano ou datas) e veja animais, interesses, adoções e o tempo médio até a adoção, doações e quem apoia, com gráficos. Os botões de exportação geram planilhas (CSV) que abrem no Excel. A aba **Histórico de operações** mostra o que foi feito na conta do abrigo.

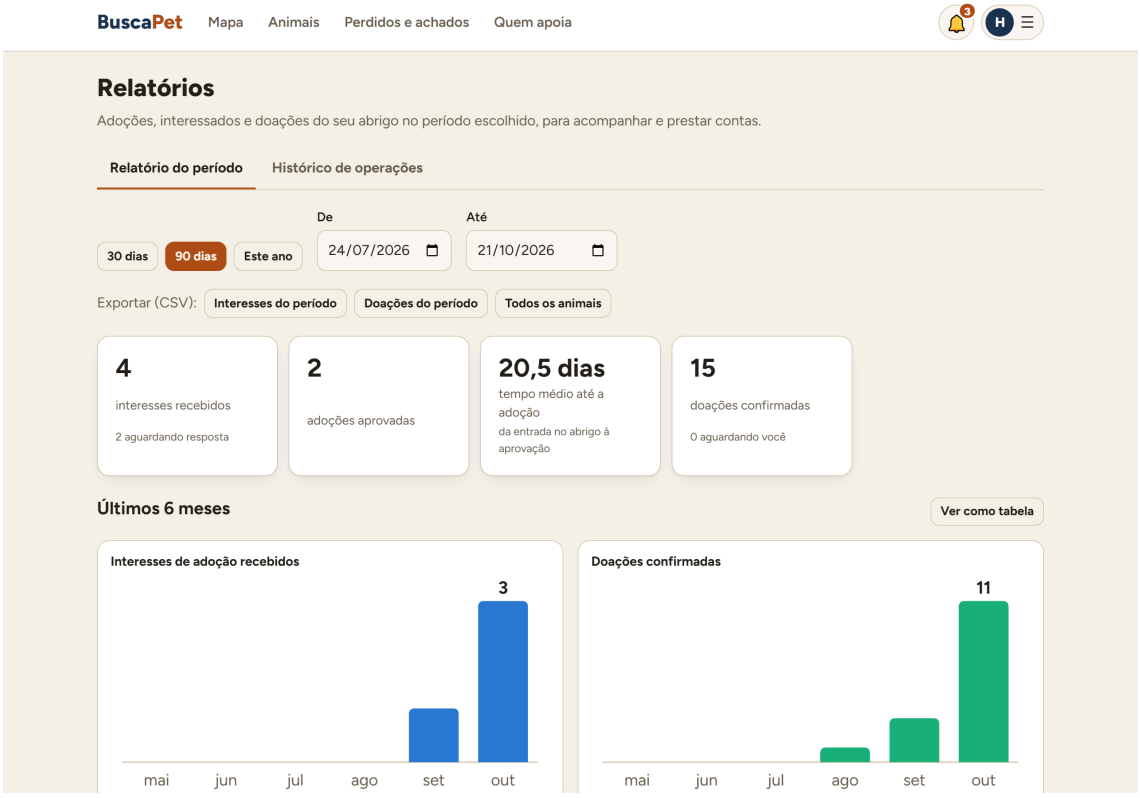


Figura 17 — Relatórios do abrigo

8. Para apoiadores

A conta de apoiador é de empresa (CNPJ). Doe pelo mural do **Quem apoia** ou pelo perfil de um abrigo; acompanhe em **Minhas doações**. O **Meu perfil público** mostra os pontos, os meses seguidos doando e os abrigos apoiados, e o selo **Apoiador ativo** aparece para quem teve doação confirmada neste mês ou no anterior.

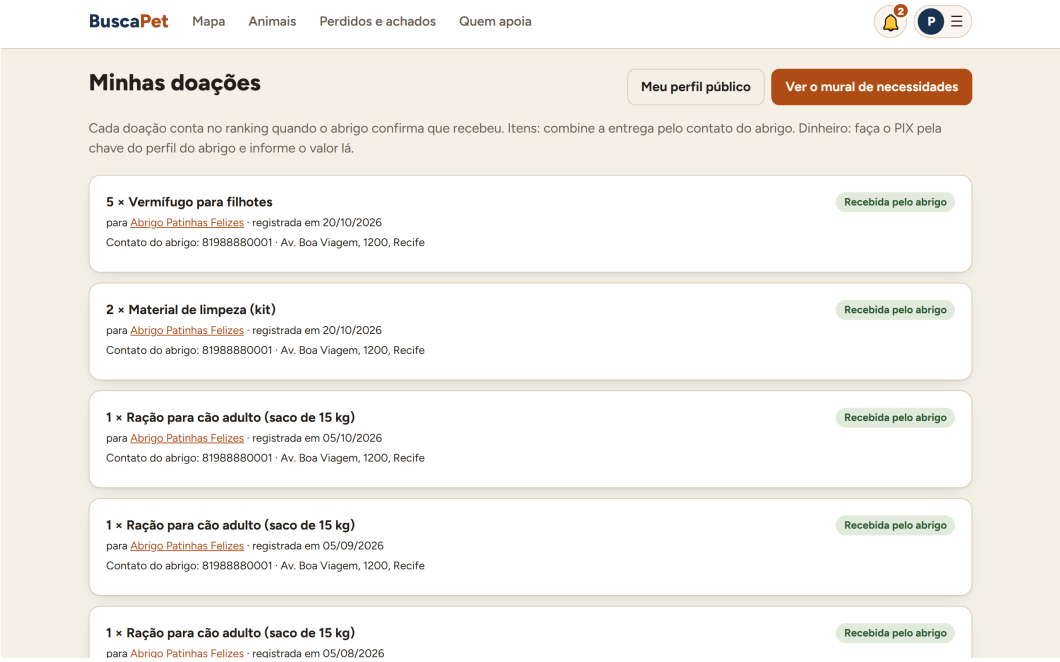


Figura 18 — Minhas doações do apoiador

9. Para a administração

- **Painel:** números do período e de agora, gráficos dos últimos 6 meses e as últimas ações.
- **Verificar abrigos: Verificar** concede o selo; **Suspender** tira o abrigo do catálogo. O abrigo é avisado.
- **Denúncias:** cada anúncio denunciado com os motivos; decida entre remover o anúncio, advertir o abrigo, banir (remove e suspende) ou ignorar.
- **Histórico de operações:** quem fez o quê e quando, com pesquisa e exportação.
- **Contas:** pesquisa por nome, e-mail, tipo e situação.
- **Núcleo OpenCL:** estado do serviço que compara as fotos e o teste de desempenho (comparação feita em um núcleo do processador × em todos os núcleos ao mesmo tempo).

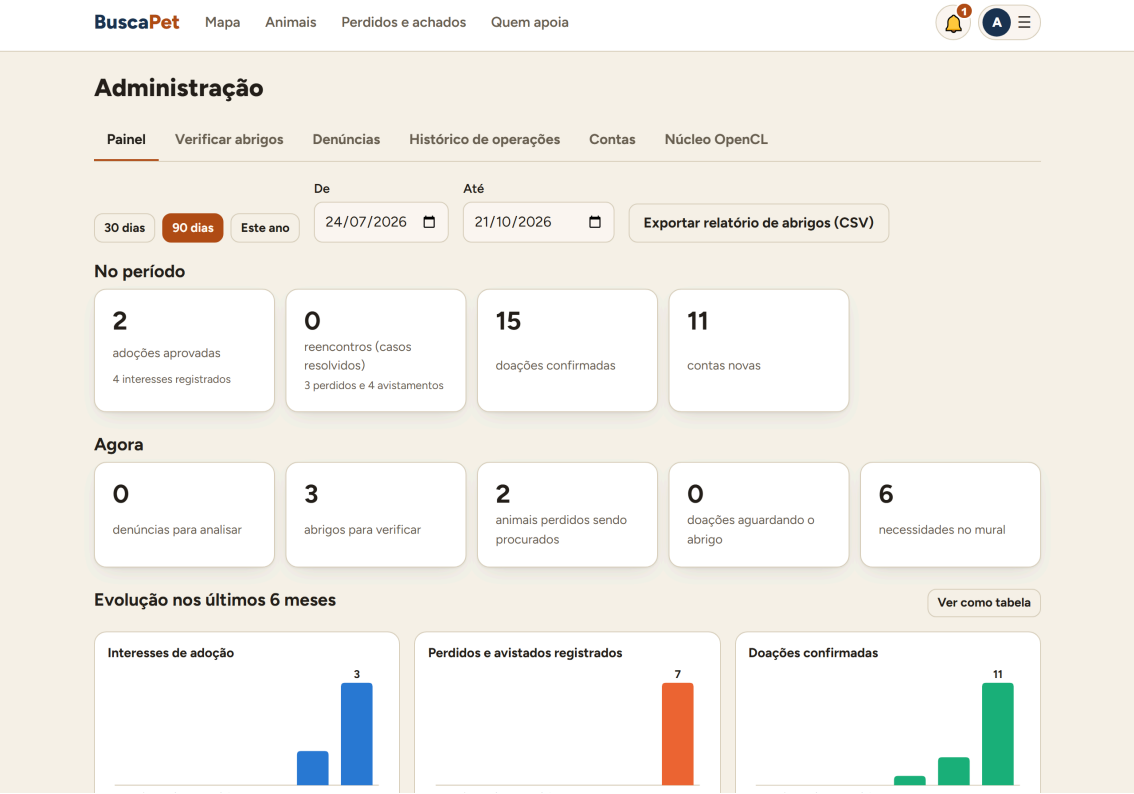


Figura 19 — Painel da administração

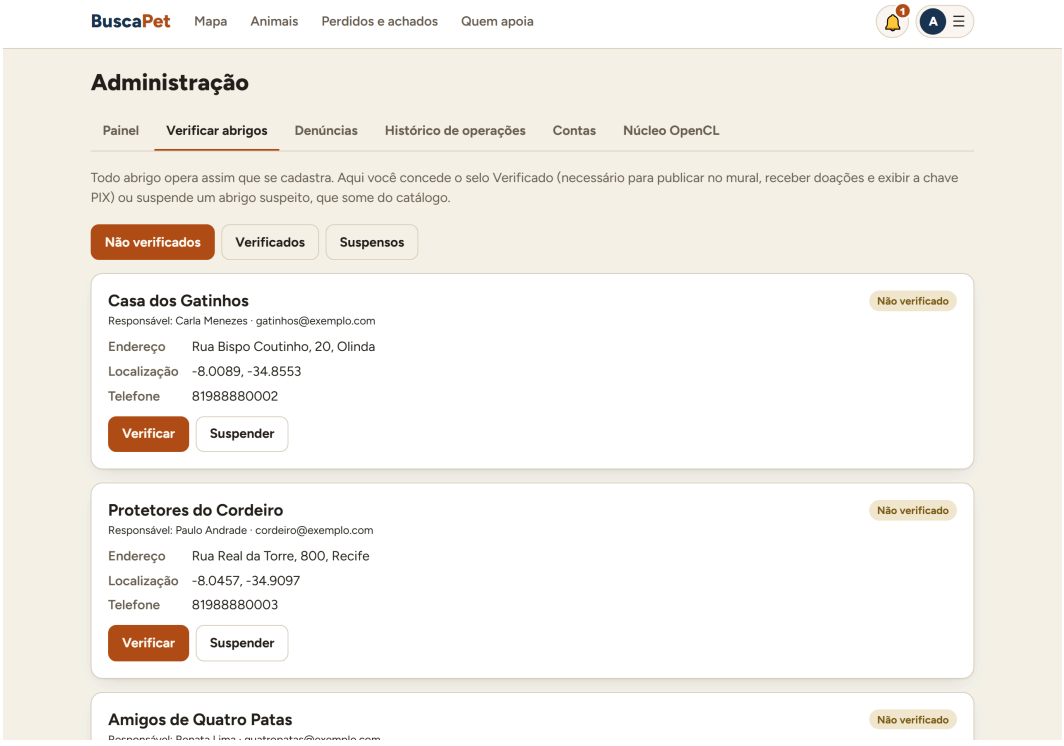


Figura 20 — Verificar abrigos

10. Mensagens, validações e situações comuns

As mensagens aparecem em faixas coloridas: **verde** quando deu certo, **amarela** para avisos e **vermelha** para erros. O campo com problema fica destacado.

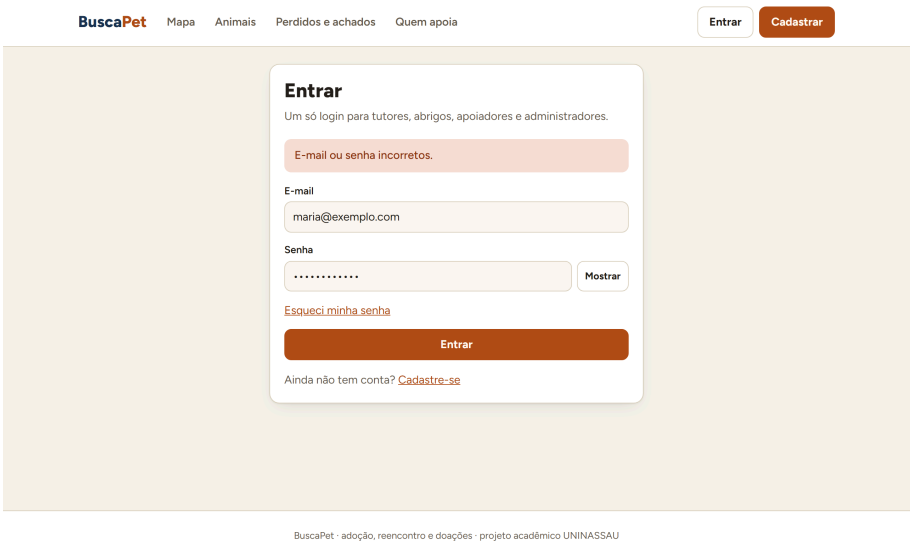


Figura 21 — Exemplo de mensagem de erro

Mensagem	O que significa / o que fazer
E-mail ou senha incorretos.	Confira o e-mail e a senha (o botão Mostrar ajuda). Esqueceu? Use "Esqueci minha senha".
Já existe uma conta com este e-mail.	Use "Entrar" ou recupere a senha.
CNPJ inválido. / Já existe um apoiador com este CNPJ.	Confira os 14 dígitos do CNPJ.
Pessoa física doa com uma conta de tutor…	A conta de apoiador é só para empresas; pessoas doam com a conta de tutor.
Escolha pelo menos uma foto. / cada foto pode ter no máximo 8 MB.	Envie de 1 a 5 fotos JPEG, PNG ou WebP, até 8 MB cada.
Este animal não está mais disponível para adoção.	Ele foi adotado ou está em processo; veja outros no catálogo.
Este abrigo ainda não tem o selo Verificado…	Só abrigos verificados recebem doações pela plataforma.
Esta necessidade já foi atendida. Veja outras no mural.	Outra pessoa completou o pedido antes; escolha outra necessidade.
Este link é inválido, já foi usado ou expirou. Peça um novo.	O link de nova senha vale 30 minutos e uma vez só.
Seu tipo de conta não tem acesso a esta página.	A página é de outro perfil (por exemplo, a área do abrigo).
Você está sem internet…	A conexão caiu; tente de novo quando voltar.
Página não encontrada	O endereço está errado ou o conteúdo foi removido; use os botões para voltar.

10.1 Perguntas frequentes

- **O BuscaPet diz que achou meu animal?** Não. Ele mostra os mais parecidos; quem reconhece é você.
- **Meu endereço aparece?** Não. Perdidos e avistados mostram um ponto aproximado ao público.
- **Por que minha doação ainda não pontuou?** Ela só conta quando o abrigo confirma que recebeu.
- **Como instalo no celular?** No rodapé, "Instalar app" (Chrome/Edge); no iPhone, pelo menu Compartilhar do Safari → "Adicionar à Tela de Início".
- **Vi um anúncio estranho (cobrança, golpe).** Na ficha do animal, "Denunciar este anúncio": a administração analisa.

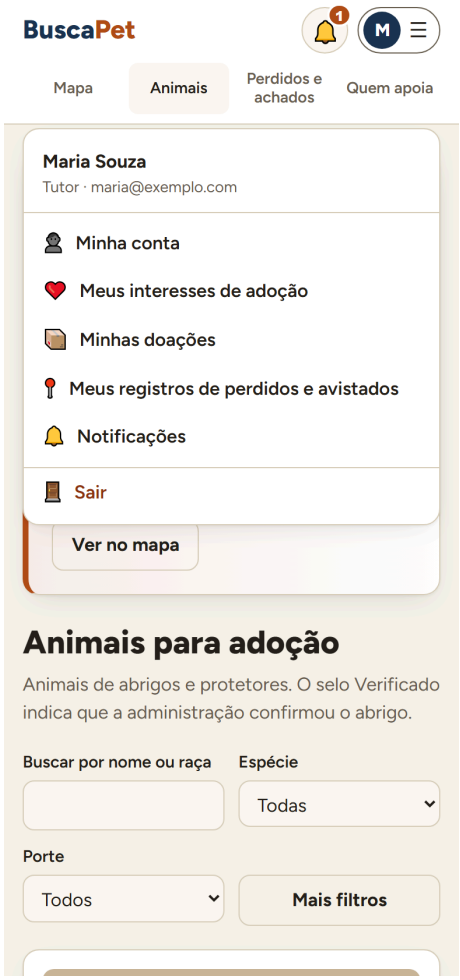


Figura 22 — No celular, o menu da conta

Universidade Maurício de Nassau
Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Manual Técnico — versão 1.0

Kaian Guthierry da Silva — 01617843
Nivaldo José de Arruda Filho — 01486768
Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. Visão geral da arquitetura
2. Tecnologias, linguagens e bibliotecas
3. Organização dos módulos e componentes
4. Estrutura do banco de dados
5. Requisitos de ambiente e dependências
6. Instalação e configuração
7. Execução
8. Variáveis de ambiente e serviços
9. APIs e integração
10. O componente de processamento (núcleo OpenCL)
11. Segurança
12. Testes e integração contínua
13. Publicação em produção
14. Manutenção e continuidade
15. Limitações conhecidas

Versão do sistema: **1.0.0-rc.1** (Release Candidate). Para quem vai instalar, executar, publicar ou dar continuidade ao desenvolvimento. O uso do sistema pelas pessoas está no **Manual do Usuário** (repositório `BuscaPet-Docs`, pasta `manuais/`).

1. Visão geral da arquitetura

O BuscaPet tem **três serviços** e um banco de dados:

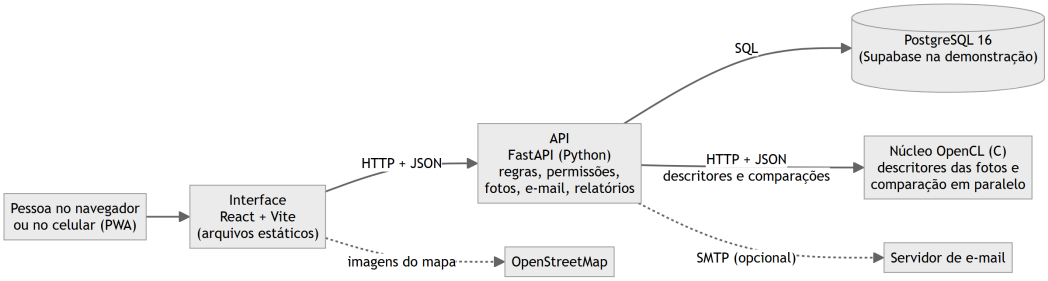


Figura 1 — Arquitetura do BuscaPet: os três serviços e o banco de dados

- **Interface** (`frontend/`): aplicação React de página única, instalável no celular (PWA). Busca todos os dados na API; só as imagens do mapa vêm direto do OpenStreetMap.
- **API** (`api/`): toda regra de negócio, autenticação (JWT), permissões por perfil, armazenamento das fotos, e-mails, relatórios e histórico de operações. É a única que acessa o banco.
- **Núcleo OpenCL** (`nucleo-opencl/`): serviço separado, em C, que transforma cada foto num **descritor** (224 números) e compara descritores em massa, em paralelo nos núcleos da CPU. A API nunca executa OpenCL: chama o núcleo por HTTP. Se o núcleo estiver fora do ar, a API continua funcionando e a busca por foto cai para a ordem por distância.
- **Banco**: PostgreSQL 16. A estrutura é criada pelas migrações do Alembic a partir dos modelos do código.

2. Tecnologias, linguagens e bibliotecas

Camada	Linguagem	Principais bibliotecas
Interface	JavaScript (React 18)	Vite 5 (build), React Router 7, Leaflet (mapas OpenStreetMap), service worker (PWA)
API	Python 3.12	FastAPI 0.158, SQLAlchemy 2, Alembic, Pydantic 2, PyJWT, bcrypt, Pillow 13 (fotos), httpx (cliente do núcleo)
Núcleo	C (C11) + OpenCL C	OpenCL 3.0 via **PoCL** (roda na CPU, sem GPU), libmicrohttpd (HTTP), cJSON
Banco	SQL	PostgreSQL 16
Testes	—	pytest + pytest-cov, Playwright (ponta a ponta), axe-core (acessibilidade), ruff (estilo)
Infraestrutura	—	Docker e Docker Compose, GitHub Actions

As versões exatas estão em `api/requirements.txt`, `frontend/package.json` e `nucleo-opencl/Dockerfile`.

3. Organização dos módulos e componentes

```
BuscaPet-App/
  api/
    app/main.py            cria a aplicação, CORS, registra os roteadores, /health
    app/core/              config (variáveis de ambiente), seguranca (senhas e JWT), deps (quem está logado e
                           permissões por perfil), fotos (validação e gravação), nucleo (cliente HTTP do núcleo),
                           semelhanca (busca e alertas), pontuacao (ranking), auditoria (histórico), email, geo
    app/routers/           uma rota por assunto: auth, usuarios, animais, interesses, abrigos, reencontro, fotos,
                           doacoes, apoiadores, notificacoes, denuncias, admin, relatorios, desempenho (núcleo)
    app/models/            tabelas (SQLAlchemy)
    app/schemas/           formatos de entrada e saída (Pydantic): validação dos dados
    app/scripts/           popular_demo, criar_admin, indexar_fotos, preparar_banco_teste, gerar_doc_api, gerar_mer
    alembic/versions/      migrações 0001 a 0005
    tests/                 testes da API (pytest)
  frontend/
    src/App.jsx            rotas das telas, cabeçalho, rodapé, proteção por perfil
    src/api.js             chamadas à API (token, erros, sem internet)
    src/auth.jsx           sessão (login, logout, conta atual)
    src/pages/             uma tela por arquivo
    src/componentes/       peças reutilizadas (menu, sino, mapa, fotos, doação, meta…)
    src/rotulos.js         textos e formatos (status, dinheiro, datas)
    e2e/                   testes de ponta a ponta (Playwright)
  nucleo-opencl/
    src/servidor.c         serviço HTTP
    src/descritor.c|h      versão sequencial (referência) dos descritores e da comparação
    kernels/               código OpenCL que roda em paralelo (descritor.cl, comparar.cl)
    bench/                 curvas de desempenho medidas
  docs/                    schema.sql, mer.md, api.md, este manual
  .github/workflows/ci.yml integração contínua
  docker-compose.yml       ambiente de desenvolvimento completo
```

Fluxo de uma requisição na API: roteador → dependências (`get_db` abre a sessão do banco; `get_usuario_atual` lê o token; `exigir_tipo(...)` confere o perfil) → validação do corpo pelo schema Pydantic → regra de negócio → `auditoria.registrar` nas ações sensíveis → `commit` → resposta no schema de saída.

4. Estrutura do banco de dados

19 tabelas e 27 chaves estrangeiras, em cinco grupos: contas e perfis (`usuarios`, `tutores`, `abrigos`, `apoiadores`, `membros_equipe_abrigo`), adoção (`animais`, `interesses`, `denuncias`), reencontro (`animais_perdidos`, `avistamentos`, `buscas_salvas`, `correspondencias`, `fotos`, `descritores_visuais`), doações (`necessidades`, `doacoes`, `pontuacoes_apoiador`) e transversais (`notificacoes`, `logs_auditoria`).

- **Diagrama:** `docs/mer.md` (completo e por módulo), gerado dos modelos.
- **SQL:** `docs/schema.sql`, conferido contra os modelos por teste, coluna a coluna.
- **Migrações:** `api/alembic/versions/` (0001 esquema inicial; 0002 alinhamento; 0003 mensagem e unicidade do interesse; 0004 anúncio removido pela moderação; 0005 doação de tutor, valor em dinheiro e meta mensal).

Convenções: chaves estrangeiras nomeadas `fk_<tabela>_<coluna>_<destino>`; datas em UTC no banco (a interface e os relatórios mostram no fuso de Recife); dinheiro em **centavos inteiros**; nada é apagado quando há histórico (conta fica `ativo = false`, anúncio ganha `removido_em`).

5. Requisitos de ambiente e dependências

Para	Precisa
API	Python **3.12** e um PostgreSQL 16 (local, Docker ou Supabase)
Interface	Node.js **20 ou mais novo**
Núcleo	**Docker** (o compilador, o PoCL e as bibliotecas ficam dentro da imagem)
Testes ponta a ponta	PostgreSQL local (cria o banco `buscapet_e2e` sozinho) e o navegador do Playwright (`npx playwright install chromium`)

Memória: com tudo rodando (Docker, API, interface e testes) uma máquina de 8 GB fica no limite; rode o ponta a ponta com `--workers=2` e desligue a API e a interface de desenvolvimento antes.

6. Instalação e configuração

```
git clone https://github.com/Equipe-BuscaPet/BuscaPet-App.git
cd BuscaPet-App

# API
cd api
python -m venv .venv
.venv\Scripts\activate            # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
# crie api/.env (veja a seção 8), depois:
set DATABASE_URL=...              # bash: export DATABASE_URL=...  (o Alembic lê a variável de ambiente)
alembic upgrade head              # cria ou atualiza as tabelas
python -m app.scripts.popular_demo   # dados de demonstração (só em banco local; senha das contas: Senha1234!)
python -m app.scripts.criar_admin --nome "Seu Nome" --email voce@exemplo.com

# Interface
cd ../frontend
npm install

# Núcleo
cd ..
docker compose up -d --build nucleo-opencl
```

Alternativa com tudo em containers: crie o `.env` na raiz (seção 8) e rode `docker compose up -d --build db api frontend nucleo-opencl`.

7. Execução

```
# API (pasta api/, ambiente virtual ativo)
uvicorn app.main:app --port 8000        # http://localhost:8000/docs  ·  /health/db

# Interface (pasta frontend/)
npm run dev                             # http://localhost:5173

# Núcleo
docker compose up -d nucleo-opencl      # http://localhost:8001/saude
```

Com o núcleo fora do Docker Compose, defina `NUCLEO_OPENCL_URL=http://127.0.0.1:8001` no `api/.env`. Fotos enviadas com o núcleo desligado ficam sem descritor: depois, `python -m app.scripts.indexar_fotos` (ou o botão em Administração → Núcleo OpenCL).

8. Variáveis de ambiente e serviços

Nenhuma credencial fica no repositório: os arquivos `.env` estão no `.gitignore` e fora das imagens Docker (`.dockerignore`). Valores reais ficam só no ambiente de cada máquina e no painel da hospedagem.

API (`api/.env` ou variáveis do serviço):

Variável	Obrigatória	Para quê	Exemplo
`DATABASE_URL`	sim	conexão com o PostgreSQL	`postgresql+psycopg://USUARIO:SENHA@HOST:5432/BANCO`
`JWT_SECRET`	**sim fora do banco local**	assina os logins. Com o valor de exemplo, a API **se recusa a subir** num banco que não seja local	gerar com `python -c "import secrets; print(secrets.token_urlsafe(48))"`
`NUCLEO_OPENCL_URL`	não	endereço do núcleo	`http://nucleo-opencl:8001` (padrão)
`CORS_ORIGENS`	em produção	endereços da interface que podem chamar a API, em **formato JSON**	`["https://buscapet.exemplo.app"]`
`URL_INTERFACE`	em produção	endereço da interface, usado nos links dos e-mails	`https://buscapet.exemplo.app`
`SMTP_HOST`, `SMTP_PORTA`, `SMTP_USUARIO`, `SMTP_SENHA`, `EMAIL_REMETENTE`	não	envio real de e-mail (recuperação de senha, avisos). Sem `SMTP_HOST`, os e-mails viram arquivos em `api/caixa-de-saida/`	—
`PASTA_FOTOS`	não	onde as fotos são gravadas (precisa ser um disco **persistente** em produção)	`/dados/fotos`

Variável	Obrigatória	Para quê	Exemplo
`PORT`	não	porta da API na imagem Docker (as hospedagens costumam definir sozinhas)	`8000`

Interface (no momento do build): `VITE_API_URL` = endereço público da API (padrão `http://localhost:8000`).

Núcleo: `PORTA` (padrão 8001), `NUCLEO_KERNELS` (pasta dos kernels; já definida na imagem).

9. APIs e integração

- **API do BuscaPet** (própria): 77 rotas, todas descritas em `docs/api.md` (gerado do código: quem pode usar, parâmetros, corpo, respostas e erros de cada uma) e no Swagger (`/docs`). Autenticação: `POST /auth/login` (formulário `username` = e-mail, `password`) devolve um token JWT de 24 horas, enviado depois no cabeçalho `Authorization: Bearer <token>`. Perfis: tutor, abrigo, apoiador e admin; `401` sem login, `403` sem permissão.
- **Núcleo OpenCL** (próprio): `GET /saude`, `POST /extrair-descritor`, `POST /comparar`, `GET /benchmark`; contrato em `nucleo-opencl/README.md`. Cliente na API: `api/app/core/nucleo.py`, com tempo limite e um **disjuntor**: depois de uma falha, a API passa 30 segundos sem tentar o núcleo, para as telas não ficarem esperando.
- **Serviços externos** (só infraestrutura, sem API de dados de terceiros): PostgreSQL (Supabase), imagens do mapa do OpenStreetMap (baixadas pelo navegador), servidor SMTP (opcional). O botão de WhatsApp é só um link `wa.me`.

10. O componente de processamento (núcleo OpenCL)

É a parte de **processamento paralelo** do projeto (disciplina de Tópicos Avançados), usada no reencontro de animais perdidos (RF-21 a RF-29).

- **Descritor (RF-21):** cada foto é reduzida a 128×128, tem a iluminação corrigida e o animal separado do fundo, e vira 224 números: histograma de **cor** (52 tons × 4 regiões), **textura** da pelagem (LBP, 10 classes) e **proporções** do animal. Toda a conta é feita com inteiros, por isso a versão paralela devolve exatamente os mesmos números que a sequencial.
- **Comparação (RF-22, RF-25):** qui-quadrado entre os histogramas e diferença de proporções → nota de 0 a 1 → semelhança **alta** (≥ 0,75), **média** (≥ 0,55) ou **baixa**. O sistema nunca afirma que encontrou o animal: a nota só ordena os candidatos; quem reconhece é a pessoa.
- **Onde é usado:** busca a partir de um perdido ou de um avistamento (`GET /reencontro/semelhantes`); **monitoramento contínuo** (cada foto nova é comparada com as buscas salvas e gera alerta ao tutor); alerta ao abrigo quando um animal parecido com um perdido chega. O monitoramento acontece no momento do registro, dentro da própria requisição (não há fila separada).
- **Paralelismo:** na extração, cada item de trabalho do OpenCL processa um bloco de 8 linhas da imagem; na comparação, cada item compara um par (foto buscada × candidato), e milhares de pares rodam ao mesmo tempo. Os kernels leem os dados sem cópia (`CL_MEM_USE_HOST_PTR`), já que o dispositivo é a própria CPU (PoCL).
- **Desempenho medido (RF-29):** i5-2400, 4 núcleos: busca 1 × 100.000 em 3,3×; lote 100 × 10.000 em 3,8×; extração de 1.000 fotos em 2,1× (kernel 3,1×). Com pouco volume o OpenCL perde (custo fixo); com volume, o ganho se aproxima do número de núcleos (Lei de Amdahl). A comparação fica visível em Administração → Núcleo OpenCL (`GET /admin/nucleo/benchmark`), que também confere que os dois modos dão o mesmo resultado.
- **Testes do núcleo:** `make teste` (dentro do build da imagem) confere a equivalência sequencial × OpenCL; a imagem só é gerada se passar. A CI sobe o núcleo e roda a API contra ele com fotos reais.

11. Segurança

- **Senhas** com bcrypt; login com tempo constante (não revela quais e-mails existem); recuperação de senha por link de uso único com validade de 30 minutos.
- **Tokens JWT** (PyJWT, HS256) com validade de 24 horas; o token de redefinição de senha não serve como login.
- **Permissões por perfil** em cada rota (`exigir_tipo`), conferidas por uma matriz de testes (rotas × perfis).
- **Fotos:** toda foto é aberta e regravada pelo servidor (confirma o formato, limita o tamanho, remove os metadados, inclusive a **localização GPS** do celular); nome aleatório.
- **Localização aproximada:** perdidos e avistamentos mostram ao público um ponto aproximado; o local exato só para quem registrou.
- **Redirecionamento seguro:** o retorno depois do login só aceita caminhos internos (`destinoInterno` em `frontend/src/auth.jsx`).
- **Segredos:** fora do Git e das imagens; a API recusa subir com o `JWT_SECRET` de exemplo num banco remoto.
- **Dependências:** auditadas na Sprint 11 (`pip-audit`, `npm audit`); veja a seção 15 sobre o que restou.

12. Testes e integração contínua

Comando	O que verifica
`cd api && pytest --cov=app`	regras de negócio, permissões, validações, documentação em dia (guia da API, MER, schema.sql); cobertura mínima de 90%
`cd api && ruff check .`	estilo e erros comuns
`cd frontend && npm run build`	a interface compila
`cd frontend && npm run e2e`	navegador de verdade contra a API e um PostgreSQL próprio (`buscapet_e2e`), no computador e no celular, com varredura de todas as telas e acessibilidade (axe-core)
`docker compose build nucleo-opencl`	compila e testa o núcleo (equivalência sequencial × OpenCL)

A CI (`.github/workflows/ci.yml`) roda tudo a cada Pull Request, em quatro trabalhos: `api`, `frontend`, `e2e` e `nucleo-opencl`.

13. Publicação em produção

A publicação precisa de quatro peças. O que vale para qualquer hospedagem:

Peça	Como publicar	Atenção
Banco	Supabase (PostgreSQL)	rodar `alembic upgrade head` com a `DATABASE_URL` de produção antes de subir a API; o plano gratuito **pausa** após cerca de uma semana sem uso (acessar o painel e restaurar)
API	imagem `api/Dockerfile` (já roda sem `--reload`, como usuário sem privilégios, na porta `$PORT`)	**as fotos ficam em disco** (`PASTA_FOTOS`): a hospedagem precisa de **volume persistente**, senão as fotos somem a cada nova publicação; definir `JWT_SECRET`, `CORS_ORIGENS`, `URL_INTERFACE` e `NUCLEO_OPENCL_URL`
Núcleo	imagem `nucleo-opencl/Dockerfile` (cerca de 850 MB)	precisa de memória para o PoCL (reserve pelo menos 1 GB); se ficar em rede interna da hospedagem, a API o alcança pelo nome do serviço
Interface	`npm run build` com `VITE_API_URL` = endereço da API → publicar a pasta `dist/` em hospedagem estática	as regras para recarregar páginas internas já estão no projeto: `frontend/vercel.json` (Vercel) e `public/_redirects` (Netlify, Render)

Depois da primeira publicação: `python -m app.scripts.criar_admin` (administrador) e `python -m app.scripts.indexar_fotos` (descritores das fotos). Os dados de demonstração (`popular_demo`) foram feitos para banco local e se recusam a rodar num banco remoto, porque as senhas deles são públicas. Para a apresentação, o caminho recomendado é cadastrar abrigos, animais e contas pela própria interface; se a equipe decidir usar o cenário de demonstração em produção, `--forcar` existe, e as senhas dessas contas precisam ser trocadas logo depois. Conferir `/health/db` na API, `/saude` no núcleo e a aba Administração → Núcleo OpenCL ("No ar").

14. Manutenção e continuidade

- **Mudou um modelo (tabela/coluna)?** `alembic revision --autogenerate -m "o que mudou"`, revisar o arquivo gerado, `alembic upgrade head`, atualizar `docs/schema.sql` e rodar `python -m app.scripts.gerar_mer`. O teste `test_doc_banco.py` acusa se faltar algum passo.
- **Mudou ou criou uma rota?** Escrever a docstring da função (vira a descrição no Swagger e no guia) e rodar `python -m app.scripts.gerar_doc_api`. O teste `test_doc_api.py` acusa se faltar.
- **Nova permissão:** usar `exigir_tipo(TipoConta....)` na rota e acrescentar a linha na matriz `tests/test_permissoes.py`.
- **Ações sensíveis:** registrar com `auditoria.registrar(...)` (aparecem no histórico de operações).
- **Datas:** gravar em UTC; mostrar no fuso de Recife. Consultas novas: testar também no PostgreSQL (o SQLite dos testes aceita coisas que o PostgreSQL recusa).
- **Fluxo de trabalho:** branch por tarefa, commits pequenos em português, Pull Request com a CI verde antes de integrar; versões com tag (`v1.0.0`).

15. Limitações conhecidas

Limitação	Motivo	Impacto
Pagamento não acontece dentro da plataforma; doação em dinheiro é por PIX direto ao abrigo, confirmada por ele	fora do escopo (exige intermediador de pagamento)	o abrigo confirma manualmente cada PIX
Tabela `membros_equipe_abrigo`, coluna `abrigos.documento_comprobatorio_url` e o tipo de correspondência `busca_inversa` existem no modelo, mas não têm tela	previstos na modelagem da Sprint 2; ficaram para depois da versão 1.0	nenhum: não interferem no uso
Fotos gravadas no disco do servidor da API	simplicidade; a troca por um armazenamento externo fica isolada em `app/core/fotos.py`	a hospedagem precisa de disco persistente (seção 13)
Alerta do `esbuild` (ferramenta do servidor de desenvolvimento do Vite)	a correção exige trocar o Vite 5 pelo 8	só afeta `npm run dev`; o site publicado (`npm run build`) não usa esse servidor
Núcleo testado em CPU (PoCL), não em GPU	a hospedagem e as máquinas da equipe não têm GPU dedicada	o código é OpenCL padrão e roda em GPU sem mudanças, se houver